Software Engineering Lab

Lab 37

Strategi Invalidation Cache

Pola Write, Mitigasi Stampede, dan Jitter TTL dalam Go

Lukman / lukman-ss

Dilema Terbesar Sistem Terdistribusi

Cache invalidation adalah salah satu dari tiga masalah sulit dalam ilmu komputer.

KUTIPAN KLASIK

Phil Karlton: "There are only two hard things in Computer Science: cache invalidation and naming things." (dan off-by-one errors).
Problem: cache menyimpan data lama; DB terupdate; reader melihat stale data.

MOTIVASI

Cache mempercepat read dengan menyimpan hasil query DB di memori.
Tanpa invalidasi tepat: inkonsistensi, cache stampede, dan outage.
Trade-off: freshness vs throughput pada write policies.

BATASAN LAB

Lab 37 fokus pada 3 write policies + 4 mitigasi stampede (SF, XFetch, SWR, Jitter).
Implementasi in-memory Go, tanpa Redis/PostgreSQL eksternal.
Verifikasi via unit test & demo CLI (race-clean).

Cache invalidation adalah salah satu dilema terbesar dalam arsitektur sistem terdistribusi skala modern. Cache dirancang sebagai lapisan memori kerja yang sangat cepat di depan database, bertujuan agar ribuan query per detik dapat dilayani tanpa membebani storage persisten. Namun, ketika record di database mengalami perubahan, cache harus segera diperbarui atau dihapus. Tanpa mekanisme invalidasi yang disiplin, cache akan terus menyimpan data basi, memicu inkonsistensi data di level aplikasi, serta dapat merusak integritas transaksi bisnis secara meluas.

Cache invalidation is the hardest problem because consistency is eventual, never instant.

Taksonomi Lab 37

Tiga pola write + empat mitigasi stampede dalam satu arsitektur Go.

Pola	Write Flow	Read Flow	Karakteristik
Cache-Aside	DB write → cache Delete	Cache miss → DB query → Set	Lazy, default pragmatic
Write-Through	DB write → cache Set (sync)	Cache hit	Strong read-after-write
Write-Behind	Cache Set → async queue flush	Cache hit	High write throughput, durability risk

Mitigasi	Teknik	Scope	Formula / Mekanisme
Singleflight	Request coalescing	Intra-process Go	singleflight.Group.Do(key, fn)
XFetch	Probabilistic early refresh	Per-request	-Δ·β·ln(U) > TTL_remaining
SWR	Stale serve + async revalidate	Per-key	ExpiresAt + staleDelta window

Lab 37 mengklasifikasikan strategi pengelolaan cache ke dalam dua dimensi arsitektural utama: write patterns dan stampede mitigations. Tiga pola penulisan—Cache-Aside, Write-Through, dan Write-Behind—menentukan keseimbangan antara latensi baca-tulis serta jaminan konsistensi data. Di sisi lain, empat mitigasi stampede—Singleflight, XFetch, Stale-While-Revalidate, dan TTL Jitter—bekerja sama untuk mencegah kehancuran database saat kunci panas kedaluwarsa secara mendadak di bawah lonjakan konkurensi. Seluruh komponen dimodelkan secara murni dalam Go standar tanpa dependensi luar.

Map cache strategies into write policies vs stampede mitigations matrix.

Cache-Aside (Lazy Loading) - Mekanisme & Flow

Mekanisme Get: cache miss → DB query → cache Set. Mekanisme Put: DB write → Invalidate cache.

GET FLOW (MISS)

1. cache.Get(key) → ErrCacheMiss (cache kosong)
2. db.Query(ctx, key) → fetch value dengan latency Δ = time.Since(start)
3. cache.Set(key, val, ttl, delta) → populasi cache dengan TTL
4. Return value ke reader

UPDATE FLOW (INVALIDATE)

1. db.Write(ctx, key, newVal) → commit ke store persisten
2. cache.Delete(key) → invalidate secara eksplisit
3. Reader berikutnya: miss → fetch DB → cache.Set

```
func (s *CacheAsideService) Update(ctx, key, val) error {
    s.db.Write(ctx, key, val) // 1. DB dulu
    s.cache.Delete(key)      // 2. baru invalidate
    return nil
}
```

Pola Cache-Aside atau lazy loading merupakan pendekatan paling populer pada sistem dengan beban baca tinggi. Pada alur pembacaan, aplikasi memeriksa cache terlebih dahulu; jika terjadi cache miss, aplikasi mengambil data langsung dari database lalu menyimpannya ke cache dengan TTL tertentu. Saat data diperbarui, aplikasi wajib menulis perubahan ke database terlebih dahulu sebelum menghapus kunci dari cache. Pola ini memastikan hanya data yang benar-benar dibaca yang menempati ruang memori cache.

Cache-aside: lazy populate on miss, write-through-then-invalidate on update.

Cache-Aside – Anomali & Race Condition

Stale read window dan concurrent write-read sequence.

STALE READ WINDOW

Setelah cache miss → DB query dimulai (bisa memakan waktu Δ).
Sepanjang proses tersebut, cache masih kosong.
Reader lain yang tiba di window ini juga melakukan miss → multiple DB queries.

CONCURRENT WRITE-READ RACE

T1: reader mulai DB query untuk key K
T2: writer melakukan db.Write(K, newVal) + cache.Delete(K)
T1: menyelesaikan query → cache.Set(K, OLD_VALUE) → menimpa nilai yang benar!
Reader lanjutan: cache hit pada OLD_VALUE yang sudah lama seharusnya invalid.

CONTOH SEKWENSIAL

Goroutine A: cache.Get(K) miss → mulai db.Query()
Goroutine B: Update(K, newVal) → db.Write() + cache.Delete()
Goroutine A: selesai db.Query() → cache.Set(K, OLD_VALUE)
Goroutine C: cache.Get(K) → hit OLD_VALUE (stale)

Meskipun fleksibel, pola Cache-Aside memiliki celah race condition antara thread pembaca dan thread penulis yang berjalan bersamaan. Jika reader mengalami cache miss lalu memulai query database yang lambat, sementara writer secara bersamaan menulis data baru ke database dan menghapus cache, reader yang terlambat dapat menuliskan kembali data usang ke dalam cache. Akibatnya, nilai usang tersebut akan bertahan di cache hingga masa TTL berakhir, menghasilkan anomali data inkonsisten yang berbahaya.

Naive cache-aside lets late miss-writes clobber validated cache entries.

Write-Through – Konsistensi Sinkron

Setiap Update: DB write → sync cache Set. Read berikutnya: cache hit, nilai terbaru.

WRITE FLOW

1. db.Write(ctx, key, val) → commit ke store persisten (blocking)
2. delta = time.Since(start)
3. cache.Set(key, val, ttl, delta) → update cache secara sinkron
4. Reader berikutnya: cache.Get(key) → hit nilai terbaru (tidak perlu DB query)

```
func (s *WriteThroughService) Update(ctx, key, val) error {
    start := time.Now()
    if err := s.db.Write(ctx, key, val); err != nil {
        return fmt.Errorf("db write failed: %w", err)
    }
    delta := time.Since(start)
    s.cache.Set(key, val, s.ttl, delta)
    return nil
}
```

KONTRAK KUAT

Read-after-write consistency terjamin untuk proses berikutnya.
Tidak ada stale read window pada update path.
Cache selalu berisi nilai terbaru yang pernah ditulis.

Pola Write-Through menawarkan jaminan read-after-write consistency yang sangat kuat dengan memperlakukan cache sebagai penyimpanan utama yang sinkron. Setiap operasi pembaruan menulis data ke database persisten dan segera memperbarui kunci cache terkait dalam satu alur eksekusi aplikasi secara berurutan. Hasilnya, setiap request pembacaan berikutnya dijamin langsung memperoleh data terbaru dari cache tanpa perlu query database tambahan. Pola ini mengeliminasi celah data basi yang sering terjadi pada Cache-Aside.

Write-through guarantees read-after-write via synchronous DB+cache update.

Write-Through – Biaya Latensi & Partial Failure

Database write penalty dan risiko kegagalan sebagian (partial failure).

BIAYA LATENSI

Write latency = DB_write_time + Cache_set_time (dua operasi blocking berurutan)
Jika DB latency naik, write latency ikut naik secara linier.
Tidak cocok untuk write-heavy workload dengan throughput tinggi.

PARTIAL FAILURE SCENARIO

1. db.Write berhasil commit ke database
2. cache.Set gagal (mis. memory pressure, node eviction)
3. State: DB terupdate, cache kosong/stale
4. Reader berikutnya: miss → fetch DB lama yang sudah benar (recover otomatis via TTL)

Failure Point	Deteksi	Recovery	Impact
DB write fail	Error return	Retry caller	Write lost
Cache set fail	Silent (swallowed)	TTL safety net	Temporary miss
Both succeed	-	-	0 impact, consistent

Kelemahan mendasar Write-Through terletak pada penalti latensi tulis dan risiko kegagalan parsial di lingkungan terdistribusi. Latensi setiap operasi update membengkak karena harus menunggu dua operasi I/O sinkron secara berturut-turut: penulisan database dan pembaruan cache. Selain itu, jika database commit berhasil namun penulisan cache gagal akibat kendala memori, status cache menjadi inkonsisten. Dalam skenario ini, TTL bertindak sebagai jaring pengaman agar data tidak terkunci dalam status usang selamanya.

Write-through doubles sync write cost; cache-set failures degrade gracefully via TTL.

Write-Behind (Write-Back) - Throughput Ekstrem

Cache Set langsung, enqueue async queue, flushWorker background ke DB.

WRITE FLOW

1. cache.Set(key, val, ttl, delta) → update cache secara langsung (synchronous)
2. select { case writeQueue <- req: | default: } → enqueue ASYNC
3. flushWorker goroutine: writeQueue <- range → db.Write() di background
4. Reader berikutnya: cache.Get() → hit nilai terbaru (zero DB round-trip)

```
func (s *WriteBehindService) Update(key, val string) {
    s.cache.Set(key, val, s.ttl, 1*time.Millisecond)
    select {
    case s.writeQueue <- WriteRequest{Key: key, Value: val}:
    default: // queue full — drop (demo limitation)
    }
}
```

THROUGHPUT GAIN

Write latency ≈ cache.Set time (milidetik), bukan DB round-trip.
flushWorker bisa memproses batch flush untuk efisiensi lebih lanjut.
Reader selalu mendapatkan nilai terbaru dari cache, bukan dari DB.

Write-Behind atau Write-Back memprioritaskan throughput penulisan ekstrem dengan memutus ketergantungan langsung terhadap latensi database persisten. Saat pembaruan terjadi, aplikasi langsung memperbarui cache di memori dan memasukkan tugas penulisan ke dalam antrean asinkron, lalu segera merespons pemanggil. Goroutine background secara berkala menguras antrean dan mengeksekusi penulisan ke database di latar belakang. Dengan mekanisme ini, latensi tulis ditekan hingga mendekati nol milidetik, sangat cocok untuk beban tulis masif.

Write-behind decouples write latency by buffering to async queue; cache stays fresh.

Write-Behind – Risiko Durabilitas & Data Loss

Queue crash, queue overflow, dan data loss saat process termination.

CRASH SCENARIO

Process mengalami crash (panic, OOM, kill) sebelum flushWorker mengeksekusi writeQueue.
Item dalam queue yang belum di-flush = data loss.
Cache (in-memory) juga hilang → key tidak dapat dipulihkan.

QUEUE OVERFLOW

select-default: bila queue penuh, write request di-drop secara silent.
Tidak ada sinyal kirim ke caller bahwa write gagal.
Pada demo: 10 writes dengan buffer size 2 → hanya 3 yang diproses.
Sisa 7 write hilang tanpa jejak.

MITIGASI PRODUKSI

Persistent queue (Kafka, WAL, file log) sebelum flush ke DB.
Acknowledge semantics: caller tunggu hingga flushWorker meng-ack.
Overflow handling: retry, backpressure, atau dead-letter queue.

Kecepatan luar biasa Write-Behind harus dibayar dengan kompromi serius pada aspek durabilitas data. Jika proses aplikasi mengalami crash, panic, atau kehabisan memori sebelum worker latar belakang sempat mengosongkan antrean, seluruh data pembaruan yang masih tertahan di antrean akan hilang permanen. Selain itu, jika antrean penuh di bawah beban tinggi, implementasi yang menggunakan select-default akan membuang request penulisan secara diam-diam. Sistem produksi wajib menggunakan antrean persisten seperti Kafka atau WAL.

Write-behind trades write durability for latency; use persistent queues in production.

Thundering Herd / Cache Stampede

N concurrent requests pada hot key expiry → N DB queries → DB overload.

MEKANISME STAMPEDE

1. Hot key K kadaluarsa (TTL habis serentak)
2. N concurrent goroutine melakukan cache.Get(K) → semua miss
3. Setiap goroutine menjalankan db.Query(K) secara paralel
4. DB menerima N query identik → CPU & connection pool terbebani
5. Latency naik berhampa → hit rate jatuh, beban makin buruk (cascading failure)

Workload	Concurrency N	DB Query Count (Naive)
Local dev	3	3
Staging	50	50
Production hot key	10,000	10,000 (DB overload)

Cache stampede atau thundering herd terjadi ketika kunci cache populer yang sering diakses kedaluwarsa secara mendadak. Puluhan hingga ribuan goroutine pembaca yang datang bersamaan akan mendapati cache miss, lalu serentak mengeksekusi query database yang identik. Fenomena ini melipatgandakan beban database sebesar perkalian laju kedatangan request dan durasi query. Akibatnya, connection pool terkuras habis, waktu tunggu melonjak tajam, dan sistem dapat mengalami kegagalan berantai hingga ketersediaan layanan runtuh sepenuhnya.

DEMO VERIFIKASI

20 concurrent goroutines → cache miss semua
NaiveStampedeService: 20 DB queries (λ·T amplification)
SingleFlightService: 1 DB query (coalescing)

Stampede amplifies concurrent misses into redundant DB query storms.

Singleflight – Request Coalescing

golang.org/x/sync/singleflight, N requests ditumpuk menjadi 1 DB call.

PRINSIP

singleflight.Group.Do(key, fn) menjamin fn dieksekusi tepat 1× per key.
Goroutine pendatang yang request key yang sama akan menunggu hasil dari goroutine pertama.
Semua waiter menerima return value yang sama (deduplication barrier).

```
import "golang.org/x/sync/singleflight"

type SingleFlightService struct {
    cache  *MemoryCache
    db     *MockDB
    ttl    time.Duration
    flight singleflight.Group
}
```

DEMO VERIFIKASI

20 concurrent goroutines → cache.Get miss semua
flight.Do(key, fn) → fn dieksekusi 1× → db.Query 1×
20 goroutines menerima hasil yang sama: db.QueryCount() == 1

Mitigasi pertama terhadap cache stampede adalah request coalescing menggunakan paket resmi Go golang.org/x/sync/singleflight. Pola ini memastikan bahwa untuk kunci yang sama, fungsi pengambilan data ke database hanya dijalankan tepat satu kali oleh satu goroutine pemenang. Goroutine-goroutine lain yang datang bersamaan akan ditahan pada barrier internal dan menunggu selesainya eksekusi goroutine pertama. Setelah selesai, seluruh pemanggil menerima salinan hasil yang identik, memangkas puluhan query database menjadi satu panggilan tunggal.

Singleflight dedupes concurrent calls: N requests collapse to 1 DB query.

Singleflight – Kode & Mekanika Do()

Double-check cache hit di dalam closure, deduplication barrier, shared result.

```
func (s *SingleFlightService) Get(ctx, key) (string, error) {
    item, err := s.cache.Get(key) // outer cache check
    if err == nil { return item.Value, nil }
    res, err, _ := s.flight.Do(key, func() (interface{}, error) {
        // Double-check cache inside flight execution
        item, err := s.cache.Get(key)
        if err == nil { return item.Value, nil }
        start := time.Now()
        val, err := s.db.Query(ctx, key)
        if err != nil { return "", err }
        delta := time.Since(start)
        s.cache.Set(key, val, s.ttl, delta)
        return val, nil
    })
    if err != nil { return "", err }
    return res.(string), nil
}
```

Implementasi SingleFlightService menerapkan teknik double-check locking di dalam closure fungsi Do untuk efisiensi maksimal. Sebelum memulai query database yang mahal, goroutine memeriksa kembali isi cache untuk mengantisipasi kemungkinan kunci tersebut telah berhasil diisi oleh eksekusi paralel lain. Singleflight mengelola tabel pemetaan pemanggilan aktif secara internal dan membagikan return value melalui channel bersama. Namun, perlindungan ini terbatas pada level satu proses; sistem multi-pod tetap membutuhkan distributed lock seperti Redis SETNX.

MEKANIKA INTERNAL

1. singleflight.Group memelihara map[key]callEntry
2. Goroutine pertama: membuat entry, mengeksekusi fn, menyimpan hasil
3. Goroutine follow: mendaftar sebagai waiter pada channel yang sama

In-flight closure double-checks cache to avoid redundant re-fetch on coalescing.

XFetch – Probabilistic Early Expiration

Formula: -Δ · β · ln(U) > TTL_remaining, compute cost Δ terukur per request.

FORMULA XFETCH

offset = -Δ · β · ln(U)
Recompute sebelum TTL resmi ketika:
-Δ · β · ln(U) > TTL_remaining

Parameters:
- Δ = time.Since(start) pada DB query terakhir (compute cost)
- β = tuning parameter (default = 1.0, >1 = lebih agresif, <1 = lebih konservatif)
- U ~ Uniform(0,1), random draw per request

```
func ShouldRecompute(delta time.Duration, beta float64,
    ttlRemaining time.Duration, u float64) bool {
    if u <= 0 || u >= 1 { return false }
    deltaSec := delta.Seconds()
    ttlRemainingSec := ttlRemaining.Seconds()
    expiryCompute := -deltaSec * beta * math.Log(u)
    return expiryCompute > ttlRemainingSec
}
```

DISTRIBUSI

XFetch adalah algoritma mitigasi stampede probabilistik berbasis riset akademis Vattani et al. Setiap kali request membaca cache, algoritma menghitung ambang batas kedaluwarsa dini berdasarkan biaya komputasi query database terakhir, parameter tuning beta, dan bilangan acak seragam. Jika nilai komputasi probabilistik melebihi sisa TTL, request tersebut akan berinisiatif merefresh cache secara proaktif sebelum waktu kedaluwarsa resmi tiba. Pola ini menyebarkan beban pembaruan cache ke sepanjang kurva eksponensial.

XFetch probabilistically shifts cache rebuild from peak expiry to exponential spread.

Menggeser puncak rebuild dari saturasi satu titik ke distribusi exponential

XFetch – Tanda Negatif Matematika

Koreksi -ln(U) > 0 vs ln(U) < 0 bug, beta parameter menggerakkan distribusi.

FORMULA BENAR

expiryCompute = -deltaSec × beta × Log(u)
karena U ∈ (0,1) → ln(U) < 0 → -ln(U) > 0
Sehingga LHS selalu POSITIF, bisa lebih besar dari TTL_remaining.

BUG FORMULA PEDAGOGIS

expiryCompute = deltaSec × beta × Log(u) (tanpa minus!)
ln(U) ∈ (-∞, 0) untuk U ∈ (0,1), jadi LHS selalu NEGATIF
RHS = ttlRemainingSec > 0 (selalu positif)
Hasil: NEGATIF > POSITIF = selalu FALSE
Efek: early refresh TIDAK PERNAH trigger (stampede tetap terjadi).

VERIFIKASI TEST

Test 1: u=0.5 → -ln(0.5)=0.6931, Δ×0.6931=69.31ms < 200ms → False
Test 2: u=0.3 → -ln(0.3)=1.2039, Δ×1.2039=120.39ms > 50ms → True
Test 3: tanpa minus sign → hasil selalu False (bug teguh)

Penerapan matematis formula XFetch menuntut pemahaman mendalam terhadap tanda minus logaritma natural. Karena variabel U berada pada rentang nol hingga satu, nilai logaritma natural ln(U) selalu menghasilkan bilangan negatif. Penambahan tanda minus di depan rumus membuat hasil kalkulasi bernilai positif, sehingga memungkinkan perbandingan yang valid terhadap sisa TTL. Kesalahan umum dalam mengabaikan tanda minus akan menghasilkan nilai negatif permanen, menyebabkan kondisi penyegaran awal tidak pernah terpenuhi dan stampede tetap terjadi.

-ln(U) maps uniform draws into positive offsets; missing minus sign silently disables early refresh.

Stale-While-Revalidate (SWR) – Toleransi Data Basi

Fresh vs Stale window vs Hard miss, RFC 5861 Cache-Control extension.

State	Kondisi	Action	Return
Fresh	now < ExpiresAt	Return cached	Cached value
Stale-1	ExpiresAt ≤ now < staleUntil	Trigger async revalidate	Stale value
Stale-2	now ≥ staleUntil (past grace)	Synchronous fetch	New DB value (blocking)
Miss	Key not in cache	Synchronous fetch	New DB value

```
item, ok := s.cache.GetRaw(key)
now := time.Now()
if ok {
    if now.Before(item.ExpiresAt) { return item.Value, nil }       // Fresh
    staleUntil := item.ExpiresAt.Add(s.staleDelta)
    if now.Before(staleUntil) {
        s.triggerRevalidate(key)                                   // Stale-1
        return item.Value, nil
    }
}
// Hard miss / Stale-2 → synchronous fetch
val, err := s.db.Query(ctx, key)
```

Stale-While-Revalidate yang diadaptasi dari RFC 5861 menawarkan solusi performa tinggi dengan memperluas toleransi terhadap data kedaluwarsa. Sistem membagi siklus hidup cache menjadi tiga jendela: fresh di mana data disajikan langsung, stale window di mana data usang segera disajikan ke klien sementara revalidasi dipicu di latar belakang, serta hard miss di mana query database harus dieksekusi secara sinkron. Pendekatan ini menjaga latensi pembacaan tetap berada di level sub-milidetik.

SWR extends cache lifetime into a grace window while async-revalidating in background.

labs/37-cache-invalidation-strategies/internal/cache/stampede.go:197–224

SWR – Async Revalidation & Deduplikasi

Serving stale immediately, background goroutine update, revalidating map guard.

REVALIDASI ASYNC

triggerRevalidate(key): spawn single goroutine per key
go func() { db.Query → cache.Set }
Reader tidak blocking; langsung menerima stale value.
Revalidasi berlangsung di background, tidak menunggu selesai.

DEDUPLIKASI (REVALIDATING MAP)

s.mu.Lock(); if s.revalidating[key] { s.mu.Unlock(); return }
s.revalidating[key] = true; s.mu.Unlock()
// goroutine: defer delete(s.revalidating, key)
Hanya 1 goroutine yang melakukan revalidasi per key.

Keberhasilan SWR sangat bergantung pada mekanisme deduplikasi revalidasi latar belakang agar tidak memicu badai permintaan baru ke database. Tanpa pembatas, ratusan request pada data basi akan masing-masing meluncurkan goroutine revalidasi baru. SWRService mengatasi ancaman ini dengan memelihara tabel penanda revalidasi berbasis mutex; hanya satu goroutine yang diizinkan memperbarui database untuk kunci yang sama pada satu waktu. Jika revalidasi gagal, klien tetap aman menerima nilai usang sebagai fallback.

```
func (s *SWRService) triggerRevalidate(key string) {
    s.mu.Lock()
    if s.revalidating[key] { s.mu.Unlock(); return }
    s.revalidating[key] = true
    s.mu.Unlock()
    s.revalCount.Add(1)
    go func() { // async fetch & set
        defer func() {
            s.mu.Lock()
        }()
        ...db.Query → s.cache.Set...
    }()
}
```

Serve stale immediately, dedupe background revalidation to one goroutine per key.

labs/37-cache-invalidation-strategies/internal/cache/stampede.go:226-254

TTL Jitter – Anti-Sinkronisasi Expiry

Base TTL + random offset [0, maxJitter), desynchronization.

FORMULA JITTER

TTL_effective = base + random(0, maxJitter)
Implementasi: time.Duration(rand.Int63n(int64(maxJitter)))
Hasil berada di range [base, base + maxJitter)
Tiap key kadaluarsa pada waktu yang berbeda.

```
func TTLWithJitter(base, maxJitter time.Duration) time.Duration {
    if maxJitter <= 0 { return base }
    jitter := time.Duration(rand.Int63n(int64(maxJitter)))
    return base + jitter
}
```

MANFAAT DESINKRONISASI

Cross-key: 1000 key dengan base=10s, jitter=[0,5s) → expire selama 10–15s
Tidak ada lonjakan simultan expiry pada t = 10s semua sekaligus
Mengurangi probabilitas banyak key miss secara bersamaan
Bukan solusi untuk single hot key stampede (lihat slide 10)

TTL Jitter adalah teknik pertahanan sederhana namun ampuh untuk mencegah sinkronisasi waktu kedaluwarsa lintas kunci cache. Ketika ribuan kunci diisi secara bersamaan saat startup sistem atau deploy aplikasi, penggunaan TTL statis akan menyebabkan seluruh kunci tersebut kedaluwarsa pada detik yang sama persis. Dengan menambahkan offset acak pada rentang nol hingga batas maksimum jitter, masa kedaluwarsa tersebar merata, mengeliminasi lonjakan serentak pada database di masa mendatang.

Jitter desynchronizes cross-key expiries, spreading cache misses over time.

Pengujian: Cache-Aside vs Write-Through vs Write-Behind

TestCachePatterns, test output & query counters.

Test	Pattern	Assertion	Expected DB Queries
Cache-Aside Read	Cache-Aside	Miss → 1 query	1
Cache-Aside Hit	Cache-Aside	Hit → 0 query	1 (stays)
Cache-Aside Update	Cache-Aside	Write+Delete → next miss	2
Write-Through Write	Write-Through	Update sync set cache	1 (no extra query)
Write-Behind Flush	Write-Behind	Async flush after sleep	1 write async

```
[Cache-Aside]
  1st GET (cache miss)  -> DB queries: 1, val: Laptop
  2nd GET (cache hit)   -> DB queries: 1, val: Laptop
  After update & GET    -> DB queries: 2, val: Laptop Pro

[Write-Through]
  After write & GET     -> DB queries: 1, val: Laptop Pro

[Write-Behind]
  Immediate GET         -> DB queries: 0, val: Laptop Pro
  After flush & GET     -> DB queries: 1, val: Laptop Pro
```

Pengujian TestCachePatterns memvalidasi perilaku ketiga write pattern menggunakan MockDB dengan counter query atomik yang presisi. Pada Cache-Aside, query database hanya bertambah pada cache miss pertama dan tetap stabil pada hit kedua, lalu bertambah lagi setelah operasi update menghapus cache. Pada Write-Through, pembacaan pasca-update langsung mendapati cache hit tanpa query database. Pada Write-Behind, update cache terjadi seketika, dan penulisan database terbukti dieksekusi secara asinkron beberapa milidetik kemudian.

MockDB query counters make write-policy behavior deterministic and testable.

labs/37-cache-invalidation-strategies/tests/cache_test.go:13-104

Pengujian: Naive Stampede vs Singleflight

20 DB queries vs 1 DB query under 20 concurrent goroutines.

NAIVE STAMPEDE (TEST)

concurrency := 20
svc := NewNaiveStampedeService(mem, db, 1*time.Minute)
20 goroutines → svc.Get(ctx, 'hot-key') secara paralel
wg.Wait()
ASSERT: db.QueryCount() > 1 (stampede terjadi!)
RESULT: 20 concurrent → 20 DB queries

SINGLEFLIGHT (TEST)

concurrency := 20
svc := NewSingleFlightService(mem, db, 1*time.Minute)
20 goroutines → svc.Get(ctx, 'hot-key') secara paralel
wg.Wait()
ASSERT: db.QueryCount() == 1 (coalescing)
RESULT: 20 concurrent → 1 DB query, 20 correct results

```
[Naive] 20 concurrent goroutines -> 20 DB queries (stampede!)
[SF]    20 concurrent goroutines -> 1 DB queries (singleflight), 20 correct results
```

Pengujian mitigasi stampede membuktikan efektivitas Singleflight secara empiris di bawah beban konkurensi tinggi. Ketika dua puluh goroutine diluncurkan secara simultan untuk mengakses kunci panas yang belum ada di cache, implementasi Naive menghasilkan dua puluh query database paralel—memperlihatkan stampede nyata. Sebaliknya, SingleFlightService berhasil menggabungkan seluruh dua puluh panggilan tersebut menjadi tepat satu query database tunggal, dan seluruh goroutine menerima nilai yang benar tanpa terjadi data race.

Singleflight collapses concurrent misses: 20 requests → 1 DB query.

Pengujian: XFetch Deterministic Simulation

Low rand draw triggers refresh, high rand draw serves cache.

SETUP TEST

delta = 100ms, beta = 1.0, ttl = 1s
Initial: cache.Get miss → db.Query → cache.Set, db.QueryCount = 1
Two deterministic randFunc injected via SetRandFunc()

HIGH U (0.5) → NO EARLY REFRESH

remaining = 200ms, u = 0.5
-ln(0.5) = 0.6931
expiryCompute = 100ms × 0.6931 = 69.31ms < 200ms
ShouldRecompute = FALSE → serve from cache
RESULT: db.QueryCount stays 1 (cache hit)

LOW U (0.3) → TRIGGER EARLY REFRESH

remaining = 50ms, u = 0.3
-ln(0.3) = 1.2039
expiryCompute = 100ms × 1.2039 = 120.39ms > 50ms
ShouldRecompute = TRUE → recompute from DB
RESULT: db.QueryCount grows to 2 (proactive refresh)

Pengujian logika XFetch dirancang secara deterministik melalui injeksi fungsi pengacak randFunc tiruan untuk menguji kedua percabangan algoritma. Ketika generator acak menghasilkan nilai tinggi mendekati satu, formula matematis memutuskan tidak perlu recompute, sehingga data disajikan langsung dari cache. Sebaliknya, saat diinjeksi nilai acak yang sangat kecil, rumus menghasilkan nilai komputasi yang melampaui sisa TTL, memicu eksekusi query pembaruan dini ke database secara terbukti dan terukur.

Deterministic randFunc injection tests both XFetch refresh and cache-hit branches.

```
  High rand draw    -> DB queries: 1, val: fresh-data (served from cache, no early refresh)
labs/37-cache-invalidation-strategies/tests/cache_test.go:161-305
```

Pengujian: SWR & Race Detection

Immediate stale serve, async reload verified, go test -race clean.

TEST SCENARIO

TTL=20ms, staleDelta=300ms
1. Initial Get → cache.Set, return 'initial-data'
2. time.Sleep(30ms) → TTL expired, masuk stale window
3. db.SetData('key', 'updated-data')
4. Concurrent Get (10 goroutines) → semua dapat 'initial-data' (stale)
5. triggerRevalidate dispatch 1 goroutine (deduplicated!)
6. time.Sleep(50ms) → async revalidasi selesai
7. Next Get → return 'updated-data' (fresh)

RACE DETECTION

go test -race ./... → PASS (1.553s, 0 races detected)
revalidateCount atomic: hanya 1 revalidasi untuk 10 concurrent stale requests
revalidating map + mutex: benar-benar mencegah storm

```
Initial GET       -> val: v1
GET (stale)       -> val: v1  (stale served immediately)
GET (fresh)       -> val: v2  (async revalidation complete)
```

Pengujian Stale-While-Revalidate memverifikasi bahwa data usang langsung dikembalikan kepada pemanggil tanpa jeda latensi begitu TTL kedaluwarsa namun masih dalam rentang toleransi stale. Sepuluh goroutine konkuren yang membaca kunci basi secara simultan terbukti hanya memicu tepat satu proses revalidasi latar belakang berkat mekanisme deduplikasi. Pengujian race detector Go dengan flag -race lulus sempurna tanpa kebocoran data, membuktikan keandalan thread-safety implementasi SWR di lingkungan produksi.

SWR serves stale immediately while async revalidation dedupes concurrent callers.

Matriks Komparasi 7 Strategi

Tabel lengkap: Pola, Latensi Tulis, Latensi Baca, Konsistensi, Risiko.

Strategi	Latensi Tulis	Latensi Baca	Konsistensi	Risiko Utama
Cache-Aside	Sync DB Write	Cache hit (0 RTT)	Eventual read-after-write	Stampede pada miss massal
Write-Through	DB+Cache (2 sync ops)	Cache hit (0 RTT)	Strong read-after-write	Latency penulisan tinggi
Write-Behind	Cache Set + async queue	Cache hit (0 RTT)	Eventual consistency	Data loss pada crash/overflow
Singleflight	Sama	Cache hit setelah 1st miss	Eventual	Tidak proteksi cross-process
XFetch	Sama	Cache hit/proactive refresh	Eventual	Sign error bug (formulasi salah)
SWR	Sama	Stale serve (0 blocking)	Eventual	Staleness window, revalidation
TTL Jitter	Sama	Cache hit (tersebar	Eventual	Tidak proteksi single hot key

Matriks komparasi tujuh strategi menyajikan panduan evaluasi menyeluruh berdasarkan profil latensi tulis, latensi baca, tingkat konsistensi, dan risiko arsitektural utama. Tidak ada satu strategi tunggal yang sempurna untuk semua skenario; setiap pilihan merupakan kompromi teknik yang sadar. Cache-Aside unggul pada efisiensi memori, Write-Through memberikan konsistensi tertinggi dengan mengorbankan kecepatan tulis, sementara Write-Behind menawarkan performa penulisan maksimum namun menuntut mitigasi khusus terhadap risiko hilangnya data saat kegagalan sistem.

No single strategy dominates; trade write latency, read latency, consistency, risk.

Panduan Adopsi & Rekomendasi Lapangan

Decision tree, kapan pakai Cache-Aside, Write-Behind, SF, SWR.

DECISION TREE

Read-heavy + cold writes?
→ Cache-Aside (default) + TTL Jitter

Hot key + stampede observed?
→ Singleflight (single proc) OR Redis lock (multi-proc)
→ XFetch (probabilistic proactive refresh)

Tolerir staleness untuk latency ultra rendah?
→ SWR (serve stale, async revalidate)

Write-heavy + throughput kritis?
→ Write-Behind + persistent queue (Kafka/WAL)
→ Hindari jika durability kritis (financial)

Read-after-write consistency diperlukan?
→ Write-Through (accept write latency)

REKOMENDASI LAPANGAN

Cache-Aside + TTL Jitter adalah default pragmatis untuk read-heavy API.

Choose invalidation strategy by matching workload profile to tolerance for staleness and durability.

lambat).
SWR ideal untuk API yang tahan terhadap data 30–60s lama.

Panduan adopsi menetapkan pohon keputusan praktis untuk mengimplementasikan strategi cache di lingkungan produksi. Untuk sebagian besar API read-heavy, Cache-Aside yang dipadukan dengan TTL Jitter adalah pilihan default terbaik. Ketika beban lalu lintas tinggi memicu stampede pada kunci panas, pasang Singleflight untuk layanan monolitik atau kunci terdistribusi untuk arsitektur multi-pod. Terapkan SWR jika sistem menoleransi data usang demi latensi sub-milidetik, dan gunakan Write-Behind hanya jika kehilangan data non-kritis dapat diterima.

Cache invalidation: di mana performance bertemu kebenaran data.

No cache is perfect—combine lazy, coalesced, and probabilistic refresh for resilience.

Software Engineering Lab 37 - Strategi Invalidation Cache

internal/cache/store.go • internal/cache/repo.go
internal/cache/patterns.go • stampede.go
tests/cache_test.go • cmd/demo/main.go • README.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/37-cache-invalidation-strategies`

Lukman / lukman-ss

Sumber: commit eac3d80a
Test passing dan diverifikasi (Race-Clean Go 1.22).
Invarian terbukti: 20 goroutine naive → 20 DB queries; dengan SingleFlight → 1 DB query.
Cache-aside adalah default pragmatis; urutan write→invalidate kritis.
Write-through memberi read-after-write sinkron; write-behind mengorbankan durability.
Singleflight + XFetch + SWR + Jitter membentukkan defense-in-depth terhadap stampede.